

**МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ)**

Институт №8 «Компьютерные науки и прикладная математика»

Лабораторная работа №1

по курсу «Технологии параллельного программирования»

**Освоение программного обеспечения для работы с технологией
CUDA.**

Примитивные операции над векторами.

Выполнил: Ахметшин Б.Р.

Группа: М8О-403Б-22

Преподаватели: А.Ю. Морозов,
Е.Е. Заяц

Москва, 2026

Условие

Цель работы: Ознакомление и установка программного обеспечения для работы с программно-аппаратной архитектурой параллельных вычислений (CUDA).

Вариант: Поэлементное умножение векторов.

Программное и аппаратное обеспечение

GPU

Имя	NVIDIA-SMI 580.82.07
Compute Capability	7.5
Объем глобальной памяти	15637086208
Разделяемая память на блок	49152
Константная память	65536
Регистров на блок	65536
Макс. потоков на блок	(1024, 1024, 64)
Макс. размер блока	2147483647 x 65535 x 65535
Количество мультипроцессоров	40

CPU

Архитектура	x86_64
CPU op-mode(s)	32-bit, 64-bit
Address sizes	48 bits physical, 48 bits virtual
Порядок байт	Little Endian
CPU(s)	12
On-line CPU(s) list	0-11
ID производителя	AuthenticAMD
Имя модели	AMD Ryzen 5 7600 6-Core Processor
Семейство ЦПУ	25
Модель	97
Потоков на ядро	2
Ядер на сокет	6
Сокетов	1
Степпинг	2
Frequency boost	enabled
CPU max MHz	5171.0000
CPU min MHz	545.0000

RAM

64 GB

Data storage

SSD на 1TB

ОС

Ubuntu 24.04.4 LTS

Среда разработки

VS Code

Компилятор

nvcc

Метод решения

Для решения задачи используется технология CUDA, позволяющая выполнять однотипные операции над элементами векторов параллельно на GPU.

Каждый поток CUDA вычисляет результат для одного элемента выходного массива:

$$c[i] = a[i] \cdot b[i].$$

Глобальный индекс потока определяется как:

$$i = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}.$$

Если индекс выходит за пределы диапазона $[0, n)$, поток не выполняет вычисления.

Перед запуском ядра входные данные копируются с host в device, после выполнения ядра результат копируется обратно на host. Для проверки корректности вызовов CUDA API используется макрос `CSC`.

Описание программы

Программа реализована в одном файле `lab1.cu`.

Основные этапы:

1. Считывание размера `n` и двух входных векторов типа `double`.
2. Выделение памяти на GPU: `cudaMalloc(d_a, d_b, d_c)`.
3. Копирование входных данных в память устройства: `cudaMemcpy`.
4. Запуск CUDA-ядра с параметрами сетки и блока.
5. Синхронизация устройства и проверка ошибок.
6. Копирование результата на host и вывод в формате `%.10e`.

CUDA-ядро:

```
__global__ void vectorMulKernel(const double *a,
    const double *b, double *c, int n) {
```

```

int idx = blockIdx.x * blockDim.x + threadIdx.x;
if (idx < n) {
    c[idx] = a[idx] * b[idx];
}
}

```

Сборка:

```

cd lab1
make

```

Запуск:

```

./lab1 < input.txt

```

Результаты

Проверочный пример

Вход:

```

3
1 2 3
4 5 6

```

Выход:

```

4.000000000000e+00 1.000000000000e+01 1.800000000000e+01

```

Замеры:

No	n	CUDA (grid, block)	CPU, ms	GPU kernel, ms	GPU total, ms
1	10000	<<< 1, 32>>>	0.002790	0.109888	280.195262
2	10000	<<< 128, 256>>>		0.093216	277.456777
3	10000	<<< 1024, 1024>>>		0.128736	281.152839
4	100000	<<< 1, 32>>>	0.029381	0.065120	276.495043
5	100000	<<< 128, 256>>>		0.070272	284.427908
6	100000	<<< 1024, 1024>>>		0.109664	281.383196
7	1000000	<<< 1, 32>>>	0.595039	0.072160	282.053103
8	1000000	<<< 128, 256>>>		0.074752	285.034988
9	1000000	<<< 1024, 1024>>>		0.191328	286.654042

Выводы

В лабораторной работе реализована программа поэлементного умножения двух векторов с использованием технологии CUDA. Алгоритм корректно выполняет распараллеливание по элементам массива и выводит результат с требуемой точностью.

По результатам замеров видно, что время выполнения самого ядра GPU мало (порядка 0.065–0.191 мс), однако полное время GPU в текущей реализации остается почти постоянным и высоким (около 276–287 мс) для всех рассмотренных n . Это указывает на доминирование накладных расходов (выделение памяти, копирование данных, синхронизации и прочие служебные операции), а не времени вычисления ядра.

Сравнение с CPU для заполненных строк показывает, что на данных размера 10^4 , 10^5 и 10^6 текущая GPU-реализация по общему времени уступает CPU, несмотря на быстрое исполнение самого ядра. Лучшая конфигурация по времени ядра в данных замерах — «<1, 32>» для $n = 10^5$ и $n = 10^6$, а для $n = 10^4$ — «<128, 256>».

Реализация пригодна как базовая учебная версия для освоения CUDA. Для получения ускорения по полному времени требуется оптимизация обменов между host и device, а также сокращение накладных расходов запуска.

Почти одинаковое время ядра для конфигураций 1x32 и 128x256 в текущих замерах объясняется тем, что операция поэлементного умножения является очень легкой и ограничивается в основном доступом к памяти и накладными расходами запуска. При этом значения времени ядра малы и близки к уровню шумов измерения. Дополнительно важно учитывать, что без grid-stride цикла конфигурация 1x32 не покрывает весь массив для больших n , поэтому такое сравнение не полностью эквивалентно.